TELUGU DOCUMENT WRITER ASSISTANT

A Grammarly-like tool for Telugu

Panuganti Bhanu

Pappu Meghana

Patnam Jahnavi

Hemanth Sai Machi Reddy

Sentence completion - Ngram Model + HMM

Objective:

Predict the most probable next Telugu word given a prefix using probabilistic modeling.

Methodology Overview:

1. **Model Type:** Hidden Markov Model (HMM) with Trigram transitions.
2. **Assumptions:**
   - The probability of a word depends on its previous n–1 words.
   - Language is a sequence of hidden semantic states producing observed words.
3. **Formulation:**

$$P(w_n \mid w_{n-2}, w_{n-1}) = \frac{C(w_{n-2}, w_{n-1}, w_n) + 1}{C(w_{n-2}, w_{n-1}) + |V|}$$

4. **Smoothing:**

Laplace (Add-One) smoothing applied on count frequencies to handle unseen words and maintain probability mass continuity

5. **Training Data:**

1M tokenized Telugu sentences → vocabulary ≈ 900k.

Sentence completion - Ngram Model + HMM

Implementation Steps:

1. **Tokenization:** Split Telugu text into word-level tokens.
2. **Context Window:** Maintain (n–1) previous tokens per prediction.
3. **Transition Matrix:** Build frequency counts for (context → next word).
4. **Prediction Phase:**
   - Given context, compute probabilities for all possible next tokens.
   - Rank and select top-k(k=3) candidates.
5. **Evaluation Metric:**
   - Perplexity using Bigram HMM only = 20.21
   - Perplexity using HMM+Ngram on sample dataset = 2.42
   - Perplexity using HMM+Ngram on complete dataset = 2.53

Sentence completion - Ngram Model + GRU

Objective: Enhance sentence fluency and handle long-term dependencies beyond N-grams.

GRU-based Reranker

- **Purpose:** Re-score N-gram candidates using deep context understanding.
- **Model Inputs:**
  - GRU-encoded context vector
  - Candidate token embedding
  - N-gram & unigram log probabilities (meta features)
- **Architecture:**
  - Embedding → GRU Encoder → Concatenate(meta features) → MLP → Binary logit
- **Training:**
  - Objective: classify true vs. false continuations.
  - Loss: Binary Cross-Entropy
  - Optimizer: Adam (lr = 1e-3)
  - Dataset: 20000 sampled contexts (train), 5 % validation split.

Sentence completion - Ngram Model + GRU

Combined Pipeline

1. **N-gram Model →** proposes top candidate words.
2. **GRU Reranker →** assigns refined semantic scores.
3. **Fluency Controls:**
   - Temperature sampling (T = 0.8)
   - Unigram boost (α = 0.15)
   - Repetition penalty (0.85)

Sample Output:

prefix = "చంపబడిన వారిలో" **Completed sentence:** చంపబడిన వారిలో కోట కమాండర్ అయిన సెయింట్ జాన్

Step 1 — context: చంపబడిన వారిలో
- కోట -> 0.999241
- కూడా -> 0.000077
- ఆమె -> 0.000076
- అన్నాడు -> 0.000076

Selected sample: కోట

Step 2 — context: వారిలో కోట
- కమాండర్ -> 0.999651
- కూడా -> 0.000035
- ఆమె -> 0.000034
- అన్నాడు -> 0.000034

Selected sample: కమాండర్

Step 3 — context: కోట కమాండర్
- అయిన -> 0.602716
- కల్నల్ -> 0.396992
- కూడా -> 0.000031
- ఆమె -> 0.000030

Selected sample: అయిన

Spell Checker - Levenshtein Distance + LSTM

Dataset & Preprocessing:

- **Source:** telugu_word_correction_pairs.json
- **Data split:** 30k (Train), 10k (Validation), 5k (Test)
- **Each entry:** { "input": noisy_word, "target": correct_word }
- **Character-level tokenization** with <sos> and <eos> markers
- **Vocabulary size** ≈ few hundred Telugu characters

Model Architecture:

- **Encoder–Decoder (Seq2Seq) model** using Bidirectional LSTM layers
- **Attention mechanism** (Luong Attention) for better context retention
- **Embedding dimension** = 64; **Hidden units** = 128
- **Beam Search decoding** for improved predictions
- **Optimizer**: Adam (lr = 3e-4), **Loss**: Sparse Categorical Crossentropy
- Trained for 10 epochs with Early Stopping and Model Checkpoints

Spell Checker - Levenshtein Distance + LSTM

Hybrid Mechanism:

- Combines Seq2Seq predictions with Levenshtein distance filtering
- Groups vocabulary by prefixes for faster candidate selection
- Selects best correction using confidence scores from the Seq2Seq model

Evaluation Methods:

- Greedy Seq2Seq (beam width = 1)
- Beam Search (beam width = 3)
- Hybrid (Levenshtein + DL confidence)

Test Results:

- Test Accuracy: 79% | Test Loss: 0.74
- Val_Accuracy:-85.6% | Val_loss:.54

Sample Output:

- **Noisy :** ప్రణత్నించాయి
- **Target:** ప్రయత్నించాయి
- **Greedy Seq2Seq** → ప్రణ్ణనించాయి
- **Beam Search (width=3)** → ప్రణ్ణనించాయి
- **Hybrid (Levenshtein + DL)** → ప్రయత్నించాయి

Grammar Checker using heuristics

Goal:

- Automatically detect and correct grammatical errors in Telugu text — focusing on agreement, case, tense, and negation.
- POS tagging, Morphological Analysis, Dependency Parsing, Rule based grammar evaluation

Approach:

1. Input Processing:

- Sentence tokenization & POS tagging using Stanza (Telugu model)
- Morphological feature inference via custom rule-based heuristics (person, number, gender, and tense)

2. Rule Engine

- Built as modular functions for each grammar rule
- Works on dependency parse + inferred morphology
- Evaluates linguistic consistency instead of keyword matching

Rules used

RULE	Description	Technique / Example
Subject–Verb Agreement	Checks consistency in **person** & **number** between subject and verb.	Dependency relation nsubj + inferred suffix features. Ex: నేను తిన్నాము
Case / Postposition Check	Validates correct case markers (ని, కు, తో, etc.) after nouns/pronouns.	Rule on ADP following valid NOUN/PRON only.
Tense Consistency	Ensures all verbs in a clause share the same tense form.	Compare Tense feature across verbs.
Negation Rule	Detects **double negatives** using regex-based morpheme patterns.	Regex on suffixes: .*లేదు$, .*వద్దు$, .*కాదు$, etc. Ex: తినలేదు లేదు
Gender Agreement	Checks mismatch between subject gender and verb suffix.	Example: ఆమె తిన్నాడు → తిన్నది

Thankyou